# Java memory model

The **Java memory model** describes how threads in the Java programming language interact through memory. Together with the description of single-threaded execution of code, the memory model provides the semantics of the Java programming language.

The original Java memory model developed in 1995 was widely perceived as broken[1] preventing many runtime optimizations and not providing strong enough guarantees for code safety. It was updated through the Java Community Process, as Java Specification Request 133 (JSR-133), which took effect back in 2004, for Tiger (Java 5.0).[2][3]

## Context

The Java programming language and platform provide thread capabilities. Synchronization between threads is notoriously difficult for developers; this difficulty is compounded because Java applications can run on a wide range of processors and operating systems. To be able to draw conclusions about a program's behavior, Java's designers decided they had to clearly define possible behaviors of all Java programs.

On modern platforms, code is frequently not executed in the order it was written. It is reordered by the compiler, the processor and the memory subsystem to achieve maximum performance. On multiprocessor architectures, individual processors may have their own local caches that are out of sync with main memory. It is generally undesirable to require threads to remain perfectly in sync with one another because this would be too costly from a performance point of view. As a result, different threads may observe different values of the same shared data at any given time.[4]

In a single-threaded environment, it is easy to reason about code execution. The typical approach requires the system to implement *as-if-serial* semantics for individual threads in isolation. When an individual thread executes, it will appear as if all of the actions taken by that thread occur in the order they appear in the program, even if the actions themselves occur out of order.

If one thread executes its instructions out of order, then another thread might see the fact that those instructions were executed out of order, even if that did not affect the semantics of the first thread. For example, consider two threads with the following instructions, executing concurrently, where the variables x and y are both initialized to 0:

| Thread 1 | Thread 2 |
|---|---|
| x = 1; | int r1 = y; |
| y = 2; | int r2 = x; |

If no reorderings are performed, and the read of y in Thread 2 returns the value 2, then the subsequent read of x should return the value 1, because the write to x was performed before the write to y. However, if the two writes are reordered, then the read of y can return the value 2, and the read of x can return the value 0.

The Java Memory Model (JMM) defines the allowable behavior of multithreaded programs, and therefore describes when such reorderings are possible. It places execution-time constraints on the relationship between threads and main memory in order to achieve consistent and reliable Java applications. By doing this, it makes it possible to reason about code execution in a multithreaded environment, even in the face of optimizations performed by the dynamic compiler, the processor(s), and the caches.

# The memory model

For the execution of a single thread, the rules are simple. The Java Language Specification requires a Java virtual machine to observe *within-thread as-if-serial* semantics. The runtime (which, in this case, usually refers to the dynamic compiler, the processor and the memory subsystem) is free to introduce any useful execution optimizations as long as the result of the thread in isolation is guaranteed to be exactly the same as it would have been had all the statements been executed in the order the statements occurred in the program (also called program order).[5]

The major caveat of this is that *as-if-serial* semantics do not prevent different threads from having different views of the data. The memory model provides clear guidance about what values are allowed to be returned when the data is read. The basic rules imply that individual actions can be reordered, as long as the *as-if-serial* semantics of the thread are not violated, and actions that imply communication between threads, such as the acquisition or release of a lock, ensure that actions that happen prior to them are seen by other threads that see their effects. For example, everything that happens before the release of a lock will be seen to be ordered before and visible to everything that happens after a subsequent acquisition of that same lock.[6]

Mathematically, there is a partial order called the *happens-before* order over all actions performed by the program. The *happens-before* order subsumes the program order; if one action occurs before another in the program order, it will occur before the other in the *happens-before* order. In addition, releases and subsequent acquisitions of locks form edges in the happens-before graph. A read is allowed to return the value of a write if that write is the last write to that variable before the read along some path in the *happens-before* order, or if the write is not ordered with respect to that read in the *happens-before* order.

# Impact

The Java memory model was the first attempt to provide a comprehensive memory model for a popular programming language.[7] It was justified by the increasing prevalence of concurrent and parallel systems, and the need to provide tools and technologies with clear semantics for such

systems. Since then, the need for a memory model has been more widely accepted, with similar semantics being provided for languages such as C++.[8]

## See also

- Memory model (computing)
- Java concurrency

## References


1. Pugh, William (2000). "The Java memory model is fatally flawed" (https://www.cs.tufts.edu/~nr/cs257/archive/bill-pugh/jmm2.pdf) (PDF). *Concurrency: Practice and Experience*. **12** (6): 445–455. doi:10.1002/1096-9128(200005)12:6<445::AID-CPE484>3.0.CO;2-A (https://doi.org/10.1002%2F1096-9128%28200005%2912%3A6%3C445%3A%3AAID-CPE484%3E3.0.CO%3B2-A). Retrieved 15 July 2021.
2. Goetz, Brian (2004-02-24). "Fixing the Java Memory Model, Part 2" (https://www.ibm.com/developerworks/library/j-jtp03304/j-jtp03304-pdf.pdf) (PDF). *IBM*. Retrieved 2010-10-18.
3. Jeremy Manson and Brian Goetz (February 2004). "JSR 133 (Java Memory Model) FAQ" (http://www.cs.umd.edu/~pugh/java/memoryModel/jsr-133-faq.html). Retrieved 2010-10-18. "*The Java Memory Model describes what behaviors are legal in multithreaded code, and how threads may interact through memory. It describes the relationship between variables in a program and the low-level details of storing and retrieving them to and from memory or registers in a real computer system. It does this in a way that can be implemented correctly using a wide variety of hardware and a wide variety of compiler optimizations.*"
4. "Chapter 17. Threads and Locks" (https://docs.oracle.com/javase/specs/jls/se8/html/jls-17.html). *Java Language Specification, Oracle*. Retrieved July 9, 2025.
5. Manson, Jeremy. "JSR-133 FAQ" (http://www.cs.umd.edu/~pugh/java/memoryModel/jsr-133-faq.html#reordering).
6. "Chapter 17. Threads and Locks" (https://docs.oracle.com/javase/specs/jls/se8/html/jls-17.html). *docs.oracle.com*.
7. Goetz, Brian (2004-02-24). "Fixing the Java Memory Model, Part 1" (https://www.ibm.com/developerworks/library/j-jtp02244/j-jtp02244-pdf.pdf) (PDF). *IBM*. Retrieved 2008-02-17.
8. Boehm, Hans. "Threads and memory model for C++" (http://hboehm.info/c++mm/). Retrieved 2014-08-08.


## External links

- Java theory and practice: Fixing the Java Memory Model, part 1 (https://www.ibm.com/developerworks/library/j-jtp02244/j-jtp02244-pdf.pdf) - An article describing problems with the original Java memory model.
- Java theory and practice: Fixing the Java Memory Model, part 2 (https://www.ibm.com/developerworks/library/j-jtp03304/j-jtp03304-pdf.pdf) - Explains the changes JSR 133 made to the Java

memory model.

- Java Memory Model Pragmatics (transcript) (http://shipilev.net/blog/2014/jmm-pragmatics/)
- The Java Memory Model links (http://www.cs.umd.edu/~pugh/java/memoryModel/)
- Java internal structure (https://jakubstransky.com/2017/11/27/structure-of-java-virtual-machine-jvm/)
- JSR-133 webpage (http://jcp.org/en/jsr/detail?id=133)
- JSR-133 FAQ (http://www.cs.umd.edu/users/pugh/java/memoryModel/jsr-133-faq.html)
- JSR-133 implementation guide (https://web.archive.org/web/20130925080251/http://g.oswego.edu/dl/jmm/cookbook.html)

---